

Sistema de Controle Acadêmico Classroom PB

relatório-processo-release 1



Release 1 — Núcleo Acadêmico

Componentes da Equipe: Alessia Bianca, Arthur Freire, Keison Raniery, José Rafael

Objetivo da Entrega: Este relatório foca puramente no gerenciamento ágil e no trabalho em equipa dentro do Azure DevOps.

Sumário

1. Organização da Equipe.....	3
2. Quadro de Itens de Trabalho (Backlog):.....	4
3. Indicadores de Desempenho: Gráfico de Burndown.....	12

1. Organização da Equipe

A distribuição das atividades seguiu o modelo participativo de colaboração, com atribuições técnicas cruzadas para garantir que nenhum membro atuasse como um ponto único de falha (*Single Point of Failure - SPOF*).

Alessia Bianca: Ficou responsável por construir a fundação de segurança, controle de acessos e consistência cronológica do sistema. Sob sua liderança, foi desenvolvido o mecanismo core de autenticação e gerenciamento de sessão global, assegurando a validação estrita de permissões por perfil de usuário. Além disso, implementou o motor de controle do ciclo de vida dos períodos letivos, estabelecendo as travas de negócio que impedem a concorrência de múltiplos períodos abertos e blindam permanentemente os dados após o encerramento oficial.

Arthur Freire: Atuou como Gerente da Sprint 1 e foi o responsável técnico pelo desenvolvimento dos motores de consistência de horários e gestão de mutação das turmas. Na Sprint 1, liderou a equipe e implementou as regras de negócio da US07 (Validação de Choque de Horário de Professor) e da US08 (Edição e Cancelamento de Turmas), criando os algoritmos de varredura que impedem conflitos na agenda dos docentes e assegurando a persistência estruturada em arquivos locais. Na Sprint 2, deu continuidade à evolução do núcleo acadêmico ao assumir o desenvolvimento de ponta a ponta da US14 (Gestão de Alteração e Cancelamento de Turmas), blindando o sistema com travas de estado que impedem modificações em turmas vinculadas a períodos letivos já iniciados ou encerrados, além de garantir a cobertura de testes automatizados via JUnit 4.

Keison Raniery: Concentrou-se na modelagem estrutural da grade horária e no gerenciamento de recursos escolares. Ficou a seu cargo a lógica de planejamento inicial de novos blocos de tempo e o desenvolvimento do motor de publicação e oferta de turmas. Sua contribuição focou na parametrização técnica das disciplinas (vagas, salas e professores) e na construção dos algoritmos antichoques que evitam colisões de horários na agenda dos docentes e superlotação de espaços físicos.

José Rafael: No ciclo inicial, desenvolveu o núcleo da US03 (Cadastro de Cursos) e da US04 (Cadastro de Disciplinas), criando as validações e restrições de perfil que impedem códigos duplicados ou dados inválidos na persistência local. Na sequência, unificou os motores de validação ao liderar a implementação da US12 (Validação de Choque de Horário de Professor) e da US13 (Obrigatoriedade de Professor

Responsável), impedindo o agendamento duplo de docentes no mesmo período letivo e bloqueando a criação ou edição de turmas sem um professor associado. Além de integrar as funcionalidades do time sem quebras de compatibilidade, refinou o tratamento de erros na interface de linha de comando e garantiu a estabilidade do ecossistema via testes automatizados em JUnit 4.

2. Quadro de Itens de Trabalho (Backlog):

Sprint 1:

- Gerente: Arthur Freire

US/TASKS:

US01 — Cadastro de usuários no sistema e bloqueio de duplicidade

Foco: Construção do repositório físico, validação de dados duplicados, criação do padrão Factory e comandos da interface CLI de cadastro.

- Task 1709: [Dev] Implementação de Persistência com Arquivo de Objetos Serializados
 - Descrição: Desenvolver o mecanismo de leitura e escrita do arquivo binário `usuarios.dat` na classe `UsuarioRepository`. Implementar algoritmos de varredura interna nos valores do mapa (`dados.values()`) para permitir buscas abrangentes tanto por matrícula quanto por e-mail de forma intercambiável.
- Task 1710: [CLI] Comandos da Interface Textual e Barramento de Dados de Cadastro
 - Descrição: Criar e integrar os parsers de comando na camada de apresentação (como a `AuthCLI`) para capturar entradas de terminal como `cadastraraluno`, `cadastrarpessoa` ou `cadastrarcoordenador`, realizando o desmembramento correto dos argumentos de texto.
- Task 1838: [JUnit] Tratamento de Exceções Customizadas e Cobertura de Testes de Cadastro
 - Descrição: Criar a classe de exceção de serviço `UsuarioJaExisteException` herdada de `RuntimeException`. Desenvolver suíte de testes automatizados no JUnit para cobrir o fluxo feliz de cadastro e os fluxos de exceção (bloqueio por e-mail ou matrícula já existentes).
- Task 1839: [Design/Dev] Isolamento de Instanciação com Padrão Factory Method
 - Descrição: Criar a classe `UsuarioFactory` para encapsular a lógica de criação de objetos concretos (`Aluno`, `Professor`, etc.). Remover os blocos de acoplamento direto `new` de dentro do

AutenticacaoService, delegando essa responsabilidade para a fábrica de forma polimórfica.

US02 — Login no sistema e validação de perfil

Foco: Motor de autenticação de dois fatores, controle global de sessão ativa e barramento de segurança por perfil na CLI.

- Task 1713: [Dev] Motor de Autenticação e Gerenciamento de Estado de Sessão
 - Descrição: Implementar o método **realizarLogin** no **AutenticacaoService** contendo as 4 etapas de validação: checagem de campos vazios, busca abrangente no repositório, conferência de senha e validação de perfil ativo. Aplicar o padrão *Singleton* para manter o estado global encapsulado da sessão (**usuarioLogado**).
- Task 1714: [CLI] Integração de Comandos de Sessão e Filtros de Segurança por Perfil
 - Descrição: Adicionar os parsers de comando **login** **[email/matricula]** **[senha]** e **logout** na **AuthCLI**. Implementar o interceptador de segurança inicial que varre o perfil da sessão ativa e barra em tempo de execução comandos restritos (ex: impedir um **ALUNO** de disparar comandos de cadastro ou de coordenação).
- Task 1715: [JUnit] Testes de Robustez de Autenticação e Casos de Negação de Acesso
 - Descrição: Desenvolver testes unitários no **AutenticacaoServiceTest** focados no motor de login. Cobrir o caminho feliz de autenticação com sucesso e cenários de falha humana/ataque (senhas incorretas, usuários inexistentes no arquivo físico e troca de estados).
- Task 1840: [JUnit] Cobertura de Fluxos de Exceção com Dados Nulos e Vazios no Login
 - Descrição: Expandir a suíte de testes JUnit para atingir a meta de 80% de cobertura da disciplina. Adicionar casos de teste que injetam deliberadamente strings vazias, em branco, dados nulos ou com espaçamentos indevidos nos campos de login, garantindo que o sistema lance os tratamentos de erro adequados sem quebrar o terminal.

US03 — Cadastro de Cursos

- **Task 1: Modelar entidade Curso** — Criação da classe `Curso` com os atributos `codigo` e `nome`, além do `toString()` formatado para arquivo plano.
- **Task 2: Implementar cadastro de cursos** — Desenvolvimento das regras de negócio no `CursoService` para validar a duplicidade de código e restringir o acesso apenas a administradores.
- **Task 3: Implementar persistência de cursos** — Criação do mecanismo de *append* em arquivo de texto no `CursoRepository`.
- **Task 4: Criar interface CLI de cursos** — Implementação da `AdminCursoCLI` com tratamento de exceções limpas via `System.err.println()`.
- **Task 5: Criar testes de cursos** — Cobertura de testes de unidade em JUnit 4 para validar fluxos normais e exceções.

US04 — Cadastro de Disciplinas

- **Task 1: Modelar entidade Disciplina** — Modelagem da classe `Disciplina` suportando código, nome, carga horária, créditos e a lista de pré-requisitos mapeada em CSV.
- **Task 2: Implementar cadastro de disciplinas** — Injeção de validações de integridade no `DisciplinaService` (carga horária e créditos estritamente positivos) e validação de existência dos pré-requisitos.
- **Task 3: Implementar persistência de disciplinas** — Construção do isolamento de leitura e escrita do arquivo local `data/disciplinas.txt`.
- **Task 4: Criar interface CLI de disciplinas** — Integração das capturas textuais e roteamento na sub-CLI correspondente.
- **Task 5: Criar testes de disciplinas** — Escrita de testes unitários automatizados cobrindo regras de negócio e validações de erro.

US05 — Cadastro e controle de estados do período letivo

Foco: Criação do ciclo de vida do período letivo, isolamento de estados (Ativo, Inativo, Encerrado) e regras para impedir múltiplos períodos ativos simultaneamente.

- Task 1901: [Dev] Implementação de Status e Regras de Negócio de Ativação do Período
 - Descrição: Adicionar o atributo **status** (String ou Enum) na entidade **Período**. Desenvolver os métodos **ativarPeríodo(id)** e **encerrarPeríodo(id)** na camada de serviço (**PeríodoService**), incluindo a validação de segurança que bloqueia a ativação de um novo período se já existir outro com o status **Ativo** no sistema.
- Task 1902: [CLI] Comandos de Transição de Estado do Período Letivo no Terminal
 - Descrição: Implementar e mapear os parsers de comando na interface de linha de comando (**PeríodoCLI**) para suportar as operações **ativarPeríodo [id]** e **encerrarPeríodo [id]**. Incluir a checagem de perfil para garantir que apenas utilizadores autenticados como **Coordenador** ou **Administrador** alterem estes estados.
- Task 1903: [JUnit] Testes Unitários do Ciclo de Vida e Estados do Período
 - Descrição: Criar uma suíte de testes automatizados no **PeríodoServiceTest** para validar as regras de transição de estado. Cobrir cenários como: ativação e encerramento com sucesso, tentativa de ativar dois períodos ao mesmo tempo (esperando falha/exceção) e bloqueio de modificações em períodos já encerrados.

US06 — Oferta de turmas para disciplinas em períodos ativos

Foco: Criação da entidade de Turma, validação de integridade referencial com disciplinas existentes e barramento de criação em períodos não-ativos.

- Task 1904: [Dev] Estruturação da Entidade Turma e Validações de Vínculo com a Disciplina
 - Descrição: Desenvolver o modelo **Turma** contendo os identificadores relacionais. Implementar o método **ofertarTurma(...)** no **TurmaService** encarregado de verificar se a disciplina informada existe e se o período letivo correspondente está estritamente com o status **Ativo** antes de efetuar a persistência no arquivo local.
- Task 1905: [CLI] Interface e Parser de Comando para Oferta Básica de Turmas

- Descrição: Desenvolver o interpretador de texto na **TurmaCLI** para o comando de oferta de turmas no terminal. Integrar as mensagens de sucesso para o utilizador e injetar a validação de privilégios para assegurar o uso exclusivo por parte do perfil **Coordenador**.
- Task 1906: [JUnit] Cobertura de Testes para Regras de Vínculo e Validação de Período
 - Descrição: Programar cenários de teste unitário no **TurmaServiceTest** para blindar o fluxo da US06. Testar o caminho feliz de alocação, a rejeição imediata ao tentar ofertar uma turma vinculada a uma disciplina fantasma (inexistente) e o bloqueio de cadastro em períodos letivos configurados como **Inativo** ou **Encerrado**.

US07 — Validação de choque de horário de professores tasks:

Foco: Garantir a consistência temporal na alocação de docentes, impedindo a criação de turmas conflitantes para o mesmo professor no mesmo período letivo.

- Task 1: [Dev] Motor Antichoque Temporal: Desenvolver o algoritmo no TurmaService para varrer a lista de turmas existentes e disparar a ChoqueHorarioException caso o mesmo professor já esteja ocupado no horário.
- Task 2: [CLI] Tratamento de Exceções de Alocação: Atualizar o CoordenadorCLI para capturar a exceção de choque de horário e exibir um feedback claro para o usuário.
- Task 3: [JUnit] Cobertura de Testes de Colisão: Programar testes unitários focados em validar a rejeição correta de turmas em horários coincidentes e a aceitação de alocações válidas.

US08 — Edição e Cancelamento de Turmas tasks:

Foco: Estabelecer as operações fundamentais de mutação e exclusão de turmas na camada de serviço e garantir a correta sincronização dessas mudanças com a persistência em disco.

- Task 1: [Dev] Operações de Mutação no Serviço: Construir a base estrutural dos métodos editarTurma e cancelarTurma no TurmaService para alteração e remoção lógica dos registros.

- Task 2: [Persistência] Atualização Definitiva de Arquivo: Implementar o método de reescrita completa no `TurmaRepository` para garantir que o arquivo `turmas.txt` reflita as edições e deleções feitas em memória.
- Task 3: [JUnit] Testes de Alteração e Deleção: Criar cenários de testes automatizados assegurando que as turmas são devidamente apagadas ou que seus atributos (vagas, sala, horário) são editados com sucesso.

Sprint 2:

- Gerente: Alessia Bianca

US/TASKS:

User Story 09 — Gerenciamento do Status do Período Letivo

Foco: Criação do ciclo de vida e estados do período letivo, isolamento de status, regras restritivas e tratamento de concorrência de períodos ativos.

- Task 1901: [Dev] Implementação de Status e Regras de Negócio de Ativação do Período
 - Descrição: Adicionar o atributo de controle `status` (String ou Enum) na entidade de modelo `Periodo`. Desenvolver os métodos `core ativarPeriodo(id)` e `encerrarPeriodo(id)` na camada de serviço (`PeriodoService`), incluindo a validação de segurança que bloqueia a ativação de um novo período caso já exista outro registro marcado atualmente com o status `Ativo` no sistema.
- Task 1902: [CLI] Comandos de Transição de Estado do Período Letivo no Terminal
 - Descrição: Criar e mapear os parsers de comando na interface de linha de comando (`PeriodoCLI / CoordenadorCLI`) para interpretar e executar de forma textual no console as operações: `ativarPeriodo [id]` e `encerrarPeriodo [id]`. Adicionar os blocos de interceptação de perfil para garantir uso exclusivo pelos privilégios de `Coordenador` ou `Administrador`.
- Task 1903: [JUnit] Testes Unitários do Ciclo de Vida e Estados do Período
 - Descrição: Programar uma suíte de testes automatizados na classe `PeriodoServiceTest` utilizando o framework JUnit para blindar as regras de transição. Cobrir exaustivamente cenários como: ativação e encerramento operando no caminho feliz, tentativa de ativação simultânea de múltiplos períodos (esperando o disparo da exceção de negócio) e bloqueio

automático de quaisquer modificações cadastrais em períodos já encerrados.

User Story 10 — Oferta de turmas para disciplinas em períodos ativos

Foco: Criação da estrutura básica da turma, validação de integridade referencial com disciplinas existentes e barramento de criação em períodos não-ativos.

- Task 1904: [Dev] Estruturação da Entidade Turma e Validações de Vínculo com a Disciplina
 - Descrição: Desenvolver o modelo **Turma** contendo os identificadores relacionais básicos. Implementar o método **ofertarTurma(...)** no **TurmaService** encarregado de verificar se a disciplina informada existe no repositório e se o período letivo correspondente está estritamente com o status **Ativo** antes de efetuar a persistência no arquivo local.
- Task 1905: [CLI] Interface e Parser de Comando para Oferta Básica de Turmas
 - Descrição: Desenvolver o interpretador de texto na **TurmaCLI** ou **CoordenadorCLI** para o comando de oferta de turmas no terminal. Integrar as mensagens de sucesso para o usuário e injetar a validação de privilégios para assegurar o uso exclusivo por parte do perfil **Coordenador**.
- Task 1906: [JUnit] Cobertura de Testes para Regras de Vínculo e Validação de Período
 - Descrição: Programar cenários de teste unitário no **TurmaServiceTest** para blindar o fluxo de criação. Testar o caminho feliz de alocação, a rejeição imediata ao tentar ofertar uma turma vinculada a uma disciplina inexistente e o bloqueio de cadastro em períodos letivos configurados como **Inativo** ou **Encerrado**.

User Story 11 — Configuração e Limites da Turma

- Descrição de Negócio: Como Coordenador, eu quero configurar as propriedades obrigatórias de cada turma (professor, limite de vagas, horário e sala) para garantir que não haja choque de horários ou salas superlotadas.

Foco: Adicionar os atributos obrigatórios da turma (professor, vagas, sala, horário) e implementar o motor de validação para impedir choques de horários ou salas.

- Task 1907: [Dev] Implementação de Atributos Obrigatórios e Motores Antichoques
 - Descrição: Adicionar os atributos **professor**, **limiteVagas**, **horario** e **sala** na classe **Turma**. Criar a lógica computacional no **TurmaService** para varrer as turmas existentes na base de dados e impedir o salvamento se o mesmo professor já estiver alocado em outra turma no mesmo horário ou se a mesma sala já estiver ocupada por outra turma no mesmo horário. Criar as exceções customizadas **ChoqueHorarioException** e **ChoqueSalaException**.
- Task 1908: [CLI] Atualização do Comando da CLI com Atributos Técnicos
 - Descrição: Atualizar o parser na **TurmaCLI** para receber a assinatura completa com os novos argumentos obrigatórios no formato estrito: **ofertarTurma [idDisciplina] [idProfessor] [limiteVagas] [horario] [sala]**. Adicionar os blocos **catch** específicos para tratar e exibir de forma amigável no console as mensagens de erro de choque de horário ou sala.
- Task 1909: [JUnit] Testes de Robustez e Cobertura de Conflitos de Turma
 - Descrição: Criar testes automatizados no **TurmaServiceTest** focados em garantir a meta de cobertura exigida pela disciplina. Os cenários de teste devem cobrir: lançamento bem-sucedido de **ChoqueHorarioException**, lançamento de **ChoqueSalaException**, rejeição de limite de vagas com números negativos ou zero, e tratamento de campos obrigatórios nulos.

US12 — Validação de Conflito de Horário de Professor

- Task 1: Implementar validação de choque de horário — Desenvolvimento do motor algorítmico temporal no **TurmaService** para varrer a base local (turmas.txt) e impedir que um mesmo docente seja alocado em duas disciplinas no mesmo período e horário.
- Task 2: Criar testes de conflito de horário — Criação de cenários de teste específicos para validar colisões parciais, totais e cenários

permitidos (como professores diferentes no mesmo horário).

- **Task 3: Ajustar CLI para mensagens de erro — Adaptação dos blocos de captura de exceções na CoordenadorCLI e AdminCLI para exibir o cabeçalho personalizado.**

US13 — Obrigatoriedade de Professor Responsável na Turma

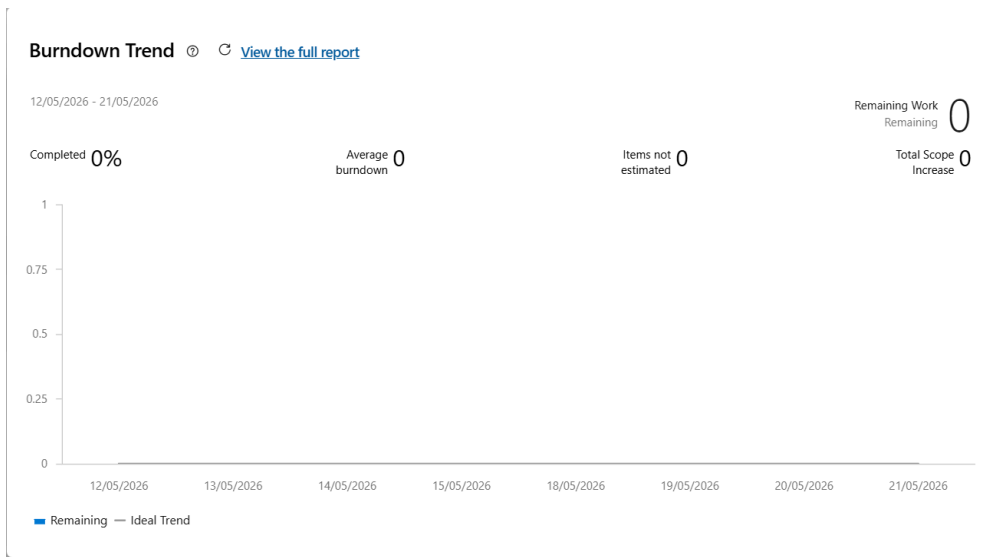
- **Task 1: Trava de consistência em criação e edição — Atualização simétrica dos métodos ofertarTurma e editarTurma no TurmaService para interceptar e rejeitar strings vazias ou nulas no campo do docente.**
- **Task 2: Desenvolver testes de obrigatoriedade — Implementação de testes em JUnit que forcem o lançamento de IllegalArgumentException caso o professor seja omitido.**
- **Task 3: Ajustar a camada de persistência — Garantia de que a escrita estruturada em arquivo plano de 6 colunas não salve registros corrompidos ou sem referência docente.**

US14 — Edição e Cancelamento de Turmas

- **Task 1: [CLI] Comandos de Terminal editarTurma e cancelarTurma: Implementar a captura e o repasse de parâmetros na interface CoordenadorCLI, exibindo feedback de sucesso e tratando exceções de forma limpa.**
- **Task 2: [Dev] Regra de Negócio e Bloqueio por Status: Desenvolver o método validarStatusPeriodo no TurmaService para impedir modificações e lançar IllegalStateException caso o período já esteja iniciado ou encerrado.**
- **Task 3: [Persistência] Lógica de Busca, Atualização e Exclusão Definitiva: Desenvolver o método atualizarArquivoCompleto no TurmaRepository para reescrever o arquivo físico turmas.txt e consolidar as alterações em disco.**
- **Task 4: [JUnit] Testes Unitários de Validação da US14: Criar cenários de testes automatizados para validar o bloqueio fora do prazo e a atualização/deleção em cenários de sucesso.**

3. Indicadores de Desempenho: Gráfico de Burndown

Sprint 1:



Sprint 2:

Burndown Trend [View the full report](#)

22/05/2026 - 01/06/2026

Completed **0%**

Average
burndown **0**

Items not
estimated **0**

Remaining Work
Remaining **0**

Total Scope
Increase **0**

